After the substrate: building software without a datacenter

Alvaro Rivera

2026-05-16

TL;DR

Building software for a small clinic, a hardware-store inventory system, or a regional logistics back office requires the operational expertise of running a datacenter — not because the domain demands it, but because the conventional cloud-microservice architecture does. The barrier is not the problem being solved; it is the stack that the problem is conventionally solved on.

This paper observes that under a journaled-program substrate — developed piece by piece across the prior six papers of this series — the datacenter ceases to be a structural requirement of running production software. Cloud and microservice ecosystems, when accessed from a journaled program, become libraries the program invokes rather than habitats the program must inhabit. Software can be shipped in shapes the conventional stack does not permit: a single on-premises appliance the customer owns, a peer mesh of devices that cooperate without a central server, a hybrid that invokes cloud services as libraries. The paper establishes the structural permission for these deployment shapes; their end-to-end operational demonstrations are engineering work the paper does not undertake.

A working instantiation makes the observation operationally inspectable: a port of the Ordering bounded context of Microsoft's `dotnet/eShop` reference application to a journaled-program substrate, replicated across three nodes that bootstrap each other through paper QR codes. After bootstrap, the cluster operates against its own journals; none of the services that originally constituted the Ordering microservice are reachable from it; the cluster continues to function under each one's absence.

A side observation, stated honestly: the role that prior literature names *server* becomes ephemeral under this substrate — discharged once per peer during the analog bootstrap, then absent from the running cluster. The construct *accidental category*, applied here at the role layer, is offered as a diagnostic lens the reader may apply to their own architecture; it is not the contribution. The contribution is the practical observation that journaled-program substrates make

datacenter-less software construction operationally inspectable, in one working case.

Thesis

Under a journaled-program substrate, the datacenter ceases to be a structural requirement of running production software. The components that the conventional cloud-microservice stack treats as habitats — database, application server, orchestration cluster, observability platform — become libraries the program invokes from its own journaled context, or fall away entirely. Software can be deployed in shapes the conventional stack does not permit: a single on-premises appliance the customer owns, a peer mesh of devices (workstations, phones) that cooperate without a central server, a hybrid that invokes cloud services as libraries. *Structurally compatible* and *operationally shipped in production* are distinct: the paper establishes the former through one working case; the latter is engineering work the paper does not undertake.

The role that prior literature names *server* — the entity that accepts authoritative writes, owns coordination, mediates between clients, and provides the address against which other parties direct their traffic — becomes ephemeral under this substrate. It is discharged once per peer during a bootstrap that can be carried by analog means and decided by humans, then absent from the running topology. This side observation connects the paper to Paper 6's lens: the server-role reads as an *accidental category* — the role-level analog of *infrastructural symptom* — once we change what nodes replicate.

Evidence. Three operating nodes derived from a port of the Ordering bounded context of Microsoft's `dotnet/eShop` reference application to a journaled-program substrate, bootstrapped by paper QR codes and authorized by human decisions. After bootstrap, the cluster operates against its own journals; none of the services that originally constituted the Ordering microservice are reachable from it; the cluster continues to function under each one's absence. The original deployment topology required a datacenter; the ported topology runs on three modest hosts and one paper QR code per peer.

Claims this paper makes

1. Under a journaled-program substrate, cloud and microservice ecosystems become libraries the program invokes rather than habitats the program inhabits; this relation is achieved through the substrate's first-class deferred-work mechanism (Reactions, Tells) plus the actor-lock's structural cost on raw I/O in the verb body, rather than through application-level resilience discipline (§2).
2. The substrate's three properties — locality, journal as substrate of the program, and self-contained runtime — combined with the actor's write-lock semantics, make the library relation the structurally cheaper path for deferred external work rather than a per-call developer discipline (§4).

3. A working instantiation, constructed by porting the Ordering bounded context of `dotnet/eShop` to a journaled-program substrate and deploying it across three nodes that bootstrap each other through paper QR codes, makes the observation operationally inspectable: after bootstrap, the cluster operates against its own journals, none of the services that originally constituted the Ordering microservice are reachable from it, and the cluster continues to function under each one's absence (§5).
4. The bootstrap of a new node — the operation that prior literature treats as the residual case requiring a persistent server — admits a presentation in which the information transferred crosses by analog means (F2) and the authorization is exercised by humans (F4), and the software portion that remains (F1, F3, F5) lasts only as long as the handshake, after which the issuer process exits (§5.3).
5. The substrate of §4 permits whole categories of small-and-medium-business software — accounting, clinic management, retail inventory, professional services — to run on hardware sized to the customer's domain, rather than on infrastructure scaled to the operator's stack. This is a structural permission, not an operational demonstration; the present paper does not exhibit an end-to-end customer-delivered appliance (§6).
6. The role that prior literature names *server* becomes ephemeral under the substrate — discharged once per peer during bootstrap, absent from the running topology thereafter — as the structural counterpart of claims 1–5. The construct *accidental category*, applied at the role layer, extends Paper 6's *infrastructural symptom* to roles; the extension is a diagnostic lens, offered as a side benefit rather than as the headline (§7).
7. The contribution is the practical observation that journaled-program substrates make datacenter-less software construction operationally inspectable in one working case; the accidental-category construct is the analytic tool through which the observation may be applied to other architectures (§7, §9).

1. Introduction

This paper extends the *infrastructural symptom* construct of Paper 6 from infrastructural layers to architectural roles, an extension that Paper 6 §9 anticipates. The genre is closer to *theory for analyzing* in the sense of Gregor (2006) than to design theory: the construct itself is Paper 6's, applied at a different level of abstraction. The novel material of this paper is the role-level demonstration — a working instantiation in which the server-role dissolves — and the operational observation that, under the same conditions, cloud ecosystems pass from architectural dependencies to libraries invoked by the program. The contribution is conceptual at the extension level and operational at the demonstration level.

Software for a small clinic, a hardware-store inventory system, a regional logistics back office: these are familiar domains. They are not technically deep — the data models are bounded, the workflows are tractable, the user counts are

modest. Yet building any of them today as production-grade software requires assembling a stack — databases, application servers, orchestration clusters, observability platforms, deploy pipelines — that the domain does not demand. The barrier is not the domain. It is the operational expertise required to run the conventional stack.

This paper observes what happens to that stack under a journaled-program substrate. Across the prior six papers of this series the substrate has been developed piece by piece: anti-porous representation (Paper 1), program-value separability (Paper 2), the pragmatic partition for deferred work (Paper 3), cross-actor causal continuity (Paper 4), the journal as substrate of the program (Paper 5), and the diagnostic lens for reading which infrastructural layers compensate for the persistence model rather than serve the problem (Paper 6). Each was argued on its own terms. This paper observes their joint operational consequence.

The consequence is that the datacenter ceases to be a structural requirement of running production software. Cloud and microservice ecosystems, when accessed from a journaled program (in the sense established in Paper 2 §1.2: the pair of domain library and journal of invocations), become libraries the program invokes rather than habitats the program must inhabit. Software can be deployed in shapes the conventional stack does not permit: a single on-premises appliance the customer owns; a peer mesh of devices (workstations, phones) that cooperate without a central server; a hybrid that invokes cloud services as libraries. The paper establishes this consequence as structural permission demonstrated through one working case (§5); end-to-end production deployment in any of these shapes is engineering work it does not undertake (§6).

The paper's argument runs in three movements. §2 names the operational relation: when external services are reached from a journaled program, what was a dependency (whose absence is an outage) becomes a library invocation (whose absence is a journaled failed call). §3 situates the paper among prior work on local-first, peer-to-peer, agent-centric, federated, and actor-model distributed systems. §4 sketches the substrate properties that make the library relation default rather than disciplined. §5 makes the observation operationally inspectable: a port of the Ordering bounded context of Microsoft's `dotnet/eShop` reference application — a widely-used example of cloud-microservice architecture in the .NET ecosystem — to a journaled-program substrate, deployed across three nodes that bootstrap each other through paper QR codes. The cluster operates against its own journals; none of the services that originally constituted the Ordering microservice are reachable from it; the cluster continues under each one's absence.

§6 examines what the observation implies for software construction in practice as a *structural permission*: software becomes deployable in shapes the conventional stack does not permit — a single appliance the customer owns, a peer mesh of devices that cooperates without a central server, a hybrid that invokes cloud services as libraries — and the barrier of operational expertise dissolves with the architectural requirement that produced it. §6 also marks the boundary

between structural permission and end-to-end operational demonstration; the present paper establishes the former, not the latter.

§7 introduces, briefly, a diagnostic lens that the reader may apply to their own architecture: the *accidental category* — a role whose necessity is traceable to a representational choice and that loses justification when the choice is replaced. The server-role of the conventional cloud-microservice stack reads as accidental under the substrate; what was a permanent fixture of the topology becomes ephemeral, discharged once per peer during analog bootstrap and absent from the running cluster thereafter. The construct extends to the role layer what Paper 6 introduced for the layer of infrastructure; it is a tool the reader may use, not the contribution of the paper.

§8 examines limitations honestly, including what the present demonstration does and does not establish. §9 concludes.

The contribution is the observation about datacenter optionality, made operationally inspectable in one working case. The construct is offered as a side benefit — the lens through which the observation can be applied to other architectures — not as the central claim. The choice of `dotnet/eShop` for the instantiation is rhetorical: it is a widely-used reference application for cloud-microservice patterns in the .NET ecosystem, whose original deployment topology makes the contrast visible. The framework underlying the instantiation was not designed to support this paper's claim; the property is identified retrospectively, after a chain of unrelated architectural decisions converged on a system in which the conventional infrastructure requirements had quietly stopped being requirements. The work is, in that sense, a forensic reading of a class of artifacts in which the property happens to hold — consistent with the analytic genre Gregor (2006) describes as *theory for analyzing*.

2. Cloud as library, not as habitat

A cluster of journaled-program nodes accesses cloud and microservice ecosystems as libraries the program invokes, not as habitats the program must inhabit. The structural difference is not in what happens when an external service is unreachable — resilience patterns developed for microservices already achieve graceful degradation under that condition — but in the substrate's relation to external invocation by default. Resilience patterns (Hystrix at Netflix; Polly in .NET; sidecar-based circuit breakers in Istio; the catalog in Nygard, 2018) achieve library-like consumption at the application level: each external call is wrapped in an opt-in policy, each fallback handler is explicitly written, and the discipline of doing this consistently is a property of the development team, not of the runtime. Under the journaled substrate, library-like consumption is the substrate's idiomatic path: external invocations made through Reactions (Paper 3) or Tells (Paper 4) are recorded as journal entries, the outcome (including failure) is part of the program's durable state, and the journal is replayable as part of the program's history. The substrate makes this path structurally

favored rather than purely disciplinary: raw I/O inside the verb body is permitted as domain code but carries a lock-cost penalty as the substrate's structural disincentive; external invocations made through Reactions and Tells are journaled by mediation through those constructs. The boundaries of this property — the lock-cost penalty on raw I/O in the verb body, the developer convention of placing slow reads in the caller's code before the verb is invoked, and the deferred-work mediation through Reactions and Tells — are stated explicitly as falsifiability conditions in §8. The journal-recorded failure is not merely observability metadata but program state available to subsequent Reactions and Tells.

The demonstration in §5 makes this concrete. After the port described in §5.1 and the bootstrap of §5.3, none of the services that previously had to be running for the Ordering microservice to exist are reachable from the cluster. The domain code runs entirely inside each node; what previously required an API host, a database, and an event bus is now a library invoked by journaled programs. The cluster operates against its own journals; the cloud topology of the original microservice has no point of contact with the cluster after the bootstrap completes.

The same relation generalizes to external services that the cluster invokes during normal operation. If the identity provider is unreachable at the moment of invocation, the journal records the failed invocation and the cluster continues. If a payment gateway is unreachable, the journal records the attempt and its outcome. The external service's absence is recorded as data, not experienced as outage. The cluster does not inhabit any of these services; it invokes them at chosen moments through journaled Reactions (Paper 3) and proceeds with their result, including the result of their absence.

This operational relation inverts the formulation that has organized the local-first software literature:

Table 1. The inversion of Kleppmann's formulation, captured as a six-word pair.

Kleppmann (Local-first)	The present observation
Own your data in spite of the cloud.	*Record the cloud; don't inhabit it.*

Local-first software treats the cloud as something the software must survive without. A journaled cluster neither survives without the cloud nor depends on it; it invokes cloud services when useful and proceeds without them when not. The two relations differ in what the absence of a cloud service means for the cluster's continued existence.

The observation shows that cloud services, when used from a journaled program, are consumed as invocable capabilities rather than as the substrate in which the program must reside.

3. Related work

Local-first software (Kleppmann et al., 2019) articulates seven ideals for software that operates primarily on devices owned by their users rather than on remote services. Replication is mediated by Conflict-free Replicated Data Types (CRDTs), which allow concurrent edits to merge without coordination. The position with respect to cloud infrastructure is precise: cloud services are admissible, but the software must continue to function in their absence. The title of the paper — *Local-First Software: You Own Your Data, in spite of the Cloud* — captures the stance: the cloud is neither rejected nor required. The server-role is reduced to one of several optional services, and its elimination is the explicit objective of the architectural pattern.

Peer-to-peer append-only logs constitute a second line. Secure Scuttlebutt (Tarr et al., 2019) replicates per-identity signed logs across a gossip network without a central coordinator; the log, not the data it carries, is the unit of synchronization. Hypercore (Holepunch, n.d.) and the Dat protocol (Ogden et al., 2017) generalize the same pattern, treating the append-only feed as a primitive replicated by a distributed hash table. In each case the unit of replication is a log of data — events, document edits, file fragments — and the application logic that interprets the log lives in the consuming application. The server's elimination is the design target of the network layer.

Holochain (Harris-Braun et al., 2018) makes the agent the source of truth. Each agent maintains a local source chain — an immutable, signed sequence of its own transactions — and contributes to a validating distributed hash table. The architecture inverts the data-centric assumption of blockchain rather than addressing the cloud directly; the elimination of the server-role is a consequence of the agent-centric inversion. The replicated artifacts are the agent's chain and the validated DHT; application logic, written as zomes, is interpreted locally by each agent.

Federated systems form a fourth line. Matrix federates an event graph across cooperating homeservers; ActivityPub federates streams of activities; the AT Protocol of Bluesky splits hosting across a Personal Data Server, a Relay, and an AppView, with account portability as the displacement mechanism. The server-role is preserved but pluralized: every participating host fulfills the role for some subset of users. The cloud is partitioned across cooperating providers rather than centralized or rejected.

Adjacent work admits brief mention. Solid (Sambra et al., 2016) proposes Personal Online Datastores that decouple identity from application providers; the server-role moves from application owner to data host without being eliminated. Earthstar (Earthstar Project, n.d.) pursues local-first synchronization without a distributed hash table, sitting between Kleppmann's CRDT model and Hypercore's gossip topology. Neither treats the server-role as a category whose referent is contingent.

Each of these lines treats the server-role as a *design target* — an architectural feature to redesign, redistribute, replace, or pluralize. None of them names the property that the present paper identifies: that the role is contingent on a choice about what nodes replicate, and that under a different choice the role has no referent at all. Table 2 summarizes the four families and the position of the present construct.

Table 2. Five families of prior work and how each treats the server-role. The bottom row identifies the construct of the present paper; the columns describe what each family replicates between nodes, where its application logic resides, why a privileged server is absent (or pluralized), and how each family situates itself with respect to cloud infrastructure.

Family	What is replicated	Where the logic lives	How the server-role is treated	Relation to the cloud
Local-first (Kleppmann)	CRDTs / data	In the application	Eliminated as a design goal	Survival without it
Hypercore / Dat / SSB	Append-only logs	In the application	Eliminated as a network goal	Replaced by peers
Holochain	Source chain + DHT	In the agent	Eliminated by agent-centric inversion	Treated as an alternative architecture
Federated (Matrix / ActivityPub / AT Protocol)	Events / activities	On federated servers	Pluralized across hosts	Fragmented across providers
The present construct	**Programs**	**In the journal of each node**	**Not a pre-requisite (accidental category)**	**Subsumed as a library**

The discriminator is the second column. What each family replicates between nodes — CRDTs and data deltas, append-only logs of data, agent source-chains, federated events and activities — determines what residual role survives. The present construct replicates neither data, deltas, logs, nor events: it replicates *programs*, in the substrate-level sense of Paper 2 §1.2 (the pair of domain library and journal of invocations). The fourth column is downstream of this choice. Under a representational choice in which what travels between nodes is the program itself, the server-role's necessity does not survive. The fifth column is downstream of the fourth: under the present construct the cloud is neither evaded, replaced, nor pluralized, but reused as a library within the program.

Beyond these four families that treat the server-role as a design target, a fifth body of work shares direct lineage with the substrate of §4 without making the server-role its design objective. Erlang/OTP (Armstrong, 2003) is the longest-running production actor system; its supervision trees and gen_server abstraction shape the ecosystem from which the substrate descends, though OTP nodes carry explicit names and gen_server processes carry explicit server-role at the level of the diagnostic lens (§7). Microsoft Orleans (Bernstein et al., 2014) introduces virtual actors with location-abstracted activation: an actor's host is determined by silo-cluster membership, not by application code. Orleans dissolves the location-of-the-actor but retains the silo cluster's coordination layer — the server-role is internalized in the membership protocol rather than externalized to the application, but it is not dissolved at the lens's reading. Akka Persistence (Lightbend, n.d.) and EventStoreDB (Event Store Ltd., n.d.) are the most direct comparators on the actor-and-event-sourcing axis: both persist and replicate the *events* an actor emits, with the actor's behavior recovered by replaying those events through code that lives outside the journal — the unit of replication is the event stream, and the program that interprets the stream is deployed alongside the runtime as application code. The present substrate inverts this relation: the unit of replication is the program itself, and what was previously called the event is recovered as the trace of the program's execution. Under the lens of §7 this difference is not a feature comparison but a representational one: events are what the program produces, not what the program *is*. Amazon Dynamo (DeCandia et al., 2007) operates as eventually-consistent without master, with conflict resolution per-key; it is the canonical AP-without-master system within the CAP positioning (Brewer, 2000; Gilbert & Lynch, 2002). Datomic (Cognitect, n.d.), a commercial immutable database, treats its journal as the source of truth but routes all writes through a single transactor — under the lens of §7, the transactor itself is an accidental category that the present substrate dissolves.

Two theoretical results bracket the structural claim. The CALM theorem (Hellerstein, 2010; Ameloot et al., 2013) establishes a formal correspondence between monotonic computation and coordination-freeness — a program admits coordination-free distributed execution iff its computation is monotonic. The substrate of §4 does not appeal to monotonicity directly, but the dissolution it documents at the role layer is consistent with CALM's prediction: when state propagation is by replay of a deterministic program over a journal rather than by reconciliation of competing writes, the conditions under which coordination becomes structurally necessary are restricted. Conflict-free Replicated Data Types (Shapiro et al., 2011), the mechanism underlying Kleppmann's local-first model, achieve coordination-free convergence at the data layer; the present substrate achieves the same property at the program layer, replicating script invocations rather than data deltas.

The substrate of §4 combines the actor model (Hewitt, 1973), event sourcing (Fowler, 2005), and CQRS (Young, 2010), synthesized as practical patterns by Vernon (2015). An additional move — domain classes carrying no persistence

concerns, discovered by reflection — antedates the synthesis by roughly a decade in the framework underlying the instantiation; that genealogy is described in Paper 6 §4.0.

This paper is the seventh in a series. Papers 1–6 develop the substrate-level foundations on which the present observation rests, culminating in Paper 6's *infrastructural symptom*, of which the diagnostic lens of §7 applies the same two indicators to architectural roles — a move Paper 6 §9 anticipates explicitly.

The contribution unique to this paper is the observation that under a journaled-program substrate, cloud and microservice ecosystems pass from architectural dependencies to library invocations (§2), with the practical consequence of making the datacenter optional for production software construction (§6). The instantiation of §5 makes the observation operationally inspectable in one working case. The diagnostic lens of §7 — *accidental category*, applied at the role layer as an extension of Paper 6's *infrastructural symptom* — is offered as a side benefit: the analytic tool through which the observation can be applied to architectures beyond the one demonstrated here.

4. The substrate that makes it possible

The substrate underlying this paper's observation is the journaled actor-native framework developed across the prior six papers of this series. Its architectural lineage is documented in Paper 6 §4.0 and is not retraced here. The substrate is described in the abstract throughout this section; its concrete instantiation, which the demonstration in §5 builds upon, is the framework whose source is publicly available (see Appendix A).

The term *program* throughout this section is used in the substrate-level sense of Paper 2 §1.2: the pair (domain library, journal of invocations) that nodes replicate to share state. The substrate's representational choice — replicating programs in this sense — is what makes it distinct from prior actor-and-event-sourcing systems. CRDT systems replicate state deltas; pub-sub and message-queue systems replicate messages; event-sourced systems (Akka Persistence, EventStoreDB, conventional CQRS+ES stacks) replicate events. The substrate of this paper replicates programs. The three properties of the substrate that follow are entailments of this choice, not three independent design decisions; together they make the cloud-as-library relation of §2 default rather than disciplined.

First, each node owns a local journal of invocations — DSL scripts (in the unit sense of Paper 2) that, on replay, reconstruct the node's state deterministically. The journal is the cumulative state-producing artifact; the state is its replay, not a captured snapshot. No external store mediates between the node and its state.

Second, the domain library — the classes and verbs that the journal scripts invoke — is loaded by reflection at startup; no node hosts the domain as a service

for others. Every node carries the same library; the substrate is symmetric across the cluster.

Third, replication is journal-to-journal: nodes exchange script references and parameters, not state diffs, and reach convergence by replaying the same sequence of scripts against the same library. The substrate's primitive is *replay* — the single primitive to which Paper 5 reduces deployment, replication, backup, and offline operation.

None of these properties requires a node to fulfill a privileged role. Each is intrinsic to the substrate, not externalized. When the cluster of §5 reaches external services, it does so against this substrate's storage: the journal records the invocation and its outcome as part of the program's history, and the cluster continues by replaying the journal. That is what makes cloud-as-library the default — the library relation is structural, not a property of how the developer wrote the call site.

The implementation source is publicly available at the commit pinned in Code Provenance; reproduction details are consolidated in Appendix A.

5. The demonstration

The observation of §2 — that under a journaled-program substrate the cloud becomes a library and the datacenter becomes optional — is made operationally inspectable by the demonstration in this section: a port of the Ordering bounded context of Microsoft's `dotnet/eShop` reference application to the substrate of §4, deployed across three nodes that bootstrap each other through paper QR codes.

5.1 The port

The instantiation ports the *Ordering* bounded context of Microsoft's `dotnet/eShop` reference application — a widely-used example of microservice-oriented cloud architecture in the .NET ecosystem — onto the substrate of §4.1. The diagnostic value of the port is what it makes visible: the lines of the original codebase that the substrate cannot host without modification are a small minority, concentrated in persistence-and-deployment glue. The aggregate, the value objects, the lifecycle verbs, and the invariants move into the journaled substrate without modification. What was previously named "the Ordering microservice" turns out to be a clean domain library wrapped in infrastructure compensating for a different substrate; the port separates the two.

5.2 The three-node deployment

Three identical processes — each running the same substrate with the ported domain library loaded by reflection — deploy across three Docker containers. The coordination mesh is fully connected: three pairwise channels forming the

complete `N(N-1)/2` graph at `N = 3`. Three is the minimum peer count at which the dissolution claim is unambiguous: with two peers, any rotation can be read as one serving the other; with three, no subset of two dominates the third.

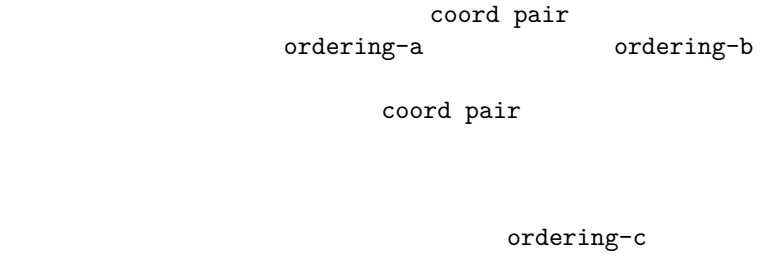

Figure 1. The three-node coordination mesh. Each node is a peer; none is privileged.

No node carries the write-acceptance privilege permanently; the role rotates symmetrically across peers. After the rotation completes, the three journals are byte-coincident.

Director rotation is the specific peer-coordination mechanism chosen for the demonstration. It is part of the StageManager pattern — a peer-to-peer coordination layer built on top of the actor substrate, located in `Choreography/StageManager/Stage.cs:470` — and is not a property of the actor model itself. The rotation is not consensus-based in the Raft (Ongaro & Ousterhout, 2014) or Paxos sense; the substrate does not elect Directors internally, run voting, maintain term or epoch numbers, or provide fencing tokens. Promotion is an external decision; the new Director's identity propagates via best-effort announce. The substrate's contribution is journal-determinism between peers (Paper 5), composable with various coordination patterns and transports; StageManager with Director rotation, over Kestrel/HTTPS, is the combination chosen for §5's Docker-container demonstration. What it does not remove, and does not claim to remove, is the broader consensus problem — a different question with different machinery.

5.3 The analog bootstrap

The bootstrap of a new peer proceeds in five phases. **F1** opens an invitation on the issuer side. **F2** transmits the invitation to the joining peer by any means, including analog ones — printed paper, a verbal dictation, an email. **F3** opens an authenticated connection back to the issuer. **F4** pauses for a human decision to approve or reject. **F5** seals a credential to the joining peer's public key and closes.

Two of the five phases are sub-software: F2 (the information transfer) and F4 (the authorization decision). Neither requires a service to be running anywhere; the carrier of F2 is whatever the operator chooses, and the decision-maker of F4

is human. The other three phases (F1, F3, F5) are themselves server-role-like discharge: the issuer listens on an address (F1), accepts a connection (F3), and seals a credential (F5). The discharge is genuine, not concealed. What §5 documents is that this discharge is ephemeral, time-bounded, and terminating: the three software-mediated phases are local to the issuer process, which exits when the bootstrap is complete. The transport stack that carries F3 (HTTPS/TLS via Kestrel; see Appendix A.4) is a property of the underlying transport, not of the journaled substrate — the substrate's contract is that whatever the transport delivers, the journal records and replays. **The issuer process does not act as a server, coordinator, authority, registry, or rendezvous point after bootstrap; its only role is to transfer a credential during the brief lifetime of the handshake, after which it ceases to exist.**

After all three containers complete the handshake — each with its own QR code carried by analog means and approved by a human — the issuer process exits. The containers continue to operate.

```
ordering-a   Up 11 seconds   0.0.0.0:6443->6443/tcp
ordering-b   Up 11 seconds   0.0.0.0:6444->6443/tcp
ordering-c   Up 11 seconds   0.0.0.0:6445->6443/tcp
```

Output of `docker compose ps` after the issuer exits. No surviving network path connects the containers to the issuer; the bootstrap papers are in a drawer.

The information that constituted the bootstrap traveled by paper. The decision that authorized it was made by a human. The cryptographic protection of the credential happened during the brief lifetime of the issuer process. The artifact that survives is the three running containers.

5.4 The empirical matrix

The demo runs across a two-by-two matrix: *in-process* versus *cross-container*, and *inline* versus *parametric*. Each cell is a complete execution of the workload across the three nodes; the in-process row reduces the bootstrap to direct invocation, while the cross-container row runs the full §5.3 protocol.

Table 3. Empirical matrix of the existence proof. Each cell reports the final journal entry count, identical on all three peers and byte-coincident on disk.

Workload regime	In-process (3 Stages, single OS process)	Cross-container (3 Dockers, real TLS + analog bootstrap)
Inline (literal scripts)	22 entries	22 entries
Parametric (Define + bind)	7 entries	7 entries

The four cells produce identical final entry counts per workload regime, and the three peers in each cell are byte-coincident on disk. The cross-container row

confirms that the operational properties of the substrate survive the addition of the analog bootstrap and the real cryptographic stack; the in-process row is the structural rehearsal that isolates the substrate's intrinsic properties from the network. The compaction from inline to parametric is real and measured; its precise ratio and the conditions under which it varies are treated in Paper 2.

Byte-coincidence is the structural witness of the headline observation: it shows that no peer is privileged — the cluster operates symmetrically, with no node fulfilling the role of authoritative writer or central coordinator. Operational metrics — write throughput, append latency, cross-DC convergence — live elsewhere in the series: Paper 5 lab L1 reports 451k events/sec on the substrate, lab L6 reports 347µs p50 append on the FS backend, and lab L4 demonstrates bit-exact cross-DC convergence under sustained workload. The present experiment is not a benchmark; it is the existence proof for the observation of §2, alongside the operational floor that Paper 5 establishes.

In the parametric regime, the journal records seven entries rather than five: each rotation Director emits its own Define + Invocation pair, because the Define cache is local to the Stage that emits it, not replicated. The journal is the catalog of each peer's declarative vocabulary; the cache's compactness lives on the Stage axis, not on the journal axis.

5.5 Partition tolerance and re-join

The same artifact additionally exhibits partition tolerance and re-join from disk. A peer can exit the cluster while the others continue operating; a fresh process can mount the absent peer's data directory and rehydrate its journal-relative state from disk alone, without contacting the issuer or the surviving peers; and the rehydrated peer integrates into the still-running cluster via peer-to-peer catch-up.

Five phases describe the sequence:

- **R1.** The three peers converge to a shared journal state.
- **R2.** One peer exits; the others continue. The cluster retains a Director and accepts new work.
- **R3.** The surviving Director issues additional work; the two surviving journals advance; the absent peer's on-disk journal is unchanged.
- **R4.** A fresh process — distinct in-memory identity, same data directory — reads the disk and reconstructs the journal-relative state of the absent peer, without contacting the issuer and without contacting the surviving peers.
- **R5.** The rehydrated peer integrates into the still-running cluster via peer-to-peer catch-up; the gap is replayed from a surviving peer; subsequent work brings all three peers back to a shared state.

The properties exercised by this scenario are not separable. The cluster's continued operation in the absent peer's absence is only possible if no node is the

cluster's source of authority. The rehydration without consulting the issuer is only possible if the disk is a self-contained witness of the peer's prior participation. The catch-up between peers is only possible if any surviving peer is capable of replaying the gap; no specialized recovery node is invoked.

Operationally, the scenario shows that a peer can leave the cluster, the cluster can continue accepting work in its absence, and the peer can re-join by reading the journal that survived its disappearance on its own disk — no central authority is consulted at any point in this loop. The role of the issuer was discharged once, at the original onboarding, and is not invoked again.

The issuer is invoked exactly once per peer. The credential sealed during the bootstrap of §5.3 is preserved across restarts, and the peer's continued participation is verified by the journal's cryptographic integrity, not by reference to a running service. If a peer loses its disk, it re-onboards as a new peer; if it preserves its disk, it returns as itself, regardless of how long it was absent.

6. What it means in practice (a structural permission)

The observation of §2 — cloud as library, not as habitat — and the demonstration of §5 together state that under a journaled-program substrate, the datacenter ceases to be a structural requirement of running production software. This section examines what that implies for software construction in practice, and what it does not.

The existing landscape. Software delivery without a datacenter is not a new category. Appliance-delivered business software (Odoo Community, Sage 50, QuickBooks Desktop, SAP Business One, Splunk Enterprise) and self-hosted applications (Nextcloud, Plex, Home Assistant, Synology DSM) deliver workloads without a datacenter, in some cases for decades. In-memory databases (VoltDB Community, SQLite, DuckDB) run application data outside cloud infrastructure. Edge-computing platforms (Cloudflare Workers, AWS IoT Greengrass) distribute logic outside the datacenter. Embedded business systems — PLCs, point-of-sale terminals, ATMs — have run software on modest hardware for forty years. The substrate of §4 does not invent any of these deployment shapes; it does not compete with the systems above; it does not claim that the categories they occupy do not exist.

What the substrate adds. The reframing is narrower than the categorical "datacenter optional" headline would suggest in isolation. The systems above each deliver software outside the datacenter, but each pays a price for that delivery: bundled database administration (Nextcloud, Sage, Odoo); no native event sourcing in most self-hosted applications; in-memory databases as storage layers without application-level semantics for replay; no journal as a unified substrate for deployment, replication, backup, and offline operation (per Paper 5). The journaled-program substrate brings the joint operational profile of cloud-native event-sourced microservices — event sourcing native (Paper 1), program-value separability for compiled cached invocations (Paper 2), the pragmatic partition

between work-due-before-responding and deferred work (Paper 3), cross-actor causal continuity through Tell (Paper 4), the journal as substrate of the four operational disciplines (Paper 5), and the diagnostic lens for reading which infrastructural layers compensate for the persistence model (Paper 6) — to hardware modest enough to fit any of the deployment shapes the systems above already inhabit. Operating an on-premises business system in the conventional stack composes several layers of operational expertise — database administration, application hosting, backup, updates, coordination between components — at the customer site or via the reseller; the journaled substrate collapses these into the substrate itself, so storage, backup, updates, and dispatch become properties of the runtime rather than concerns the operator composes. The substrate is not necessarily faster or cheaper than the existing systems — that is a separate empirical question — but its *operational surface* is structurally smaller. The unique contribution is this bundle on modest hardware; the categories of datacenter-less delivery themselves predate the substrate.

The contribution can be considered illustratively (not as evidence) through a concrete scenario. A developer or small team encounters a customer too small to be addressed economically by existing mid-market or enterprise systems and not adequately served by generic spreadsheets, single-purpose SaaS, or bespoke development. Under the conventional stack, the developer faces a ladder of operational concerns before they reach the domain: *I need a server — at the customer or in a cloud — and either way I need to provision a database, configure an application tier, set up backup, arrange monitoring, and handle updates before I can start writing the domain.* The toy alternative — a spreadsheet, a script, a one-off utility — does not survive contact with the customer's actual usage. The ladder does not begin at the domain; it begins below it. The journaled-program substrate flattens the ladder: the developer can write the domain as their first concern (a library of classes and verbs in the language they already know), while the runtime substrate handles persistence, backup, updates, and dispatch. The operational surface that the conventional stack ten-steps to is, under the substrate, one or two — and those one or two do not demand operational specializations the developer does not have. This scenario is offered for illustration; the paper makes no controlled-study claim about which developer populations are or are not currently served by existing systems, nor about which barriers (operational, marketing, support, sales, regulatory) are binding in particular cases.

New deployment shapes. The substrate is compatible with deployment shapes the conventional cloud-microservice stack does not permit. Beyond the single-appliance shape — software shippable as a deliverable artifact the customer owns, sized to the domain — the same substrate enables a peer mesh: devices that cooperate without a central server, where each peer participates as a node in the §5 sense and the cluster operates from the peers' joint awakeness rather than from a server's uptime.

The §5 demonstration is precisely this mesh at N=3 on Docker containers, com-

posing three layers: the substrate (the actor model with journaled programs replayed against a shared domain library), a peer-coordination pattern (the StageManager pattern with its Director rotation), and a transport (Kestrel/HTTPS). The substrate's core does not prescribe a particular coordination pattern or transport; StageManager and Kestrel/HTTPS are demo-specific choices appropriate to Docker. The transport-layer pluggability is exercised in the published artifact: alongside the Kestrel/HTTPS used in §5, a second transport implementation based on the SimpleX messaging protocol has been implemented and verified to run on Android, and — when substituted for Kestrel/HTTPS as the peer transport in the §5 three-node Docker scenario (an `smp-server` container providing the SMP relay; the three `ordering-*` containers connecting to it via `ConfigureTransport(TransportType.SimpleX, ...)` instead of HTTPS) — produces the same convergence outcome with bit-equivalent final journal entries to the HTTPS variant (`Choreography/Transport/SimpleX/` in the public repository at commit `b42d0f7`). The layered framing — substrate / coordination pattern / transport — is therefore not hypothetical but exercised across at least two transport implementations. Other deployment shapes compose the substrate with different layer combinations. Workstations on a corporate LAN can reuse the demo's Kestrel-based transport directly — a clinic with three workstations is the closest direct lift of the demonstration. Mobile peer meshes compose the substrate with the SimpleX-based transport (its Android viability verified at the transport layer) and a coordination pattern appropriate to opportunistic synchronization rather than continuous Kestrel sessions; the operational engineering of a full mobile peer-mesh deployment — battery management, NAT traversal between home networks, OS suspension handling, cellular reconnection patterns — lives at the operational layer downstream of transport choice and is not undertaken by the present paper. The substrate's contribution is the actor-journal-replay model; transport and coordination are downstream choices the deployment makes.

The contrast with specialized peer-to-peer systems — blockchain consensus, distributed hash tables, mesh-routing protocols — is that the substrate's mechanism (journaled program, replayed deterministically against a shared domain library) is not specialized to a particular peer-mesh problem; the same substrate composes with whatever transport-and-coordination layer the deployment requires, serving a delivery mesh of riders coordinating inventory, a small clinic where nurses share a patient roster during their shifts, a classroom of students cooperating on a shared exercise, a community game where players' devices form the world's persistent state — each with the transport and coordination layers chosen appropriately for that deployment substrate.

What remains. §2 and §5 demonstrate operationally that a journaled cluster operates without a datacenter footprint in one case — the Ordering bounded context of `dotnet/eShop` — at one scale — N=3 — under one transport — HTTPS/TLS, on Docker containers. From that demonstration, this section derives by structural argument that appliance delivery, peer-mesh deployment, and accessibility-of-development are architecturally permitted. The paper does

not demonstrate an end-to-end appliance delivery to a customer in production, a peer mesh of mobile devices in field use, a developer-to-customer flow with operational expertise lower than the conventional stack requires, or the long-tail viability of journaled software for any of the application categories listed. The structural permission is what the paper establishes; the end-to-end demonstration of each deployment shape is engineering work the paper acknowledges as outside its scope. Beyond the architectural permission, the substrate does not address sales channels (how the customer learns this software exists), distribution (how it reaches the customer), support models (who answers the phone when something fails), regulation (where data resides and whose privacy laws apply), or pricing (license vs subscription vs free). These are downstream of architecture and depend on choices the architect does not control. The shapes the developer might choose (single appliance at the customer site, peer mesh across devices, cluster across machines, hybrid with cloud invocations) are not new categories; they are deployment topologies for which polished products often do not exist or are poorly fitted at the small end of the market, and the obstacle to building those products has multiple components. This paper addresses one of those components — the architectural one. The others remain. The conditions under which empirical evidence would refute the structural permission are stated in §8.

7. A diagnostic lens, briefly: when the server-role is accidental

The substrate's three properties of §4 and the demonstration of §5 make a side observation possible: the role that prior literature names *server* — the entity that accepts authoritative writes, owns coordination, mediates between clients, provides the address against which other parties direct their traffic — becomes ephemeral rather than permanent. In the demonstration of §5, no node carries the four functions of the server-role permanently. Acceptance of authoritative writes and ownership of coordination redistribute under Director rotation; mediation and addressing distribute across the peer-to-peer mesh; partition tolerance and re-join distribute across surviving peers via catch-up (§5.5); the bootstrap issuer (§5.3) participates once per peer and exits. **The server is not a permanent identity. It is a transferable coordinator role any peer can assume in turn — and any peer can leave the cluster and re-join without reference to a coordinator outside it.**

At the level of construct, this is Paper 6's *infrastructural symptom* lens applied at the role layer: the server-role reads as an *accidental category* — the role-level expression of infrastructural symptom — when its necessity is traceable to a specific representational choice (Paper 6's *contingency* indicator at the role layer) and disappears when that choice is replaced (Paper 6's *dissolution* indicator). The reading operates per role-in-use-case; many roles remain structural under any substrate — identity providers backed by cryptographic root authority, gateways mediating external systems, computationally expensive services —

and no substrate change makes them otherwise. The full apparatus of the lens — formal indicators, the constructibility bound on counterfactual substrates, per-use-case discrimination — is in Paper 6 §3 and is not retraced here. Appendix B catalogs seven canonical architectural roles under the two indicators, as the role-level analog of Paper 6's Table 1, for readers who want to apply the lens to systems beyond the one demonstrated in §5.

The lens is offered as the analytic tool through which the consequence of §2 and §6 can be applied to architectures beyond the demonstration of §5 — not as the contribution of the paper.

8. Limitations and counter-arguments

The demonstration of §5 carries specific limitations that follow from the choices made for the demo. They are reported here honestly; none of them affects the observation of §2 or its practical consequence in §6, but each affects what the present experiment alone establishes.

Identity in the port is local. The original `dotnet/eShop` deployment uses an external identity provider; the port substitutes a local string-typed identifier. The lens of §7 does not read identity providers as dissolving — identity providers backed by cryptographic root authority are structural requirements, as Appendix B notes — and the demo does not exhibit one. A second experiment that integrates an external identity provider as a Reaction-invoked library would extend the empirical surface without changing the lens's reading.

Cross-context dependencies are not portrayed. The Ordering bounded context of `dotnet/eShop` carries references resolved by the Catalog and Basket bounded contexts; the port treats those references as opaque identifiers. The same port mechanism applies to Catalog and Basket; the present experiment exhibits Ordering only.

The two limitations above warrant clarification, because they are easy to read as selection bias when they are in fact bounded-context isolation in the Domain-Driven Design sense (Evans, 2003; Vernon, 2013). The Ordering bounded context, in any architecture — microservices, monolith, or journaled-program substrate — references Catalog and Basket entities by identifier rather than by full object, because cross-context object resolution is itself an integration boundary, not a property of Ordering's domain. Identity, similarly, lives in its own bounded context (the Identity microservice in the original eShop); Ordering does not authenticate users, it accepts a `UserId`. Removing those from the port is not eliminating components that "need a server"; it is respecting the same boundaries DDD prescribes. Appendix B itself lists identity providers backed by cryptographic root authority as structural requirements — the lens never reads identity as dissolving, so its exclusion is not adverse evidence.

The empirical matrix is at N = 3. The mesh in §5.2 connects three peers — the minimum non-trivial fully-connected mesh — using a specific peer-

coordination mechanism (Director rotation) chosen for the demonstration. The substrate's structural claim is independent of node count and of the specific coordination mechanism: it is that journaled programs replicate without a privileged coordinator at any scale at which the journal-determinism property holds between peers. Whether the specific Director rotation mechanism demonstrated at N=3 scales operationally to larger N is engineering work the paper does not undertake; other coordination mechanisms (gossip, hierarchical overlays, sharding) are well-understood in distributed systems and could be used in conjunction with the substrate's journal-determinism at larger scales, but the present paper does not demonstrate any such combination. The substrate's structural permission holds at any N where journal-determinism between peers is preserved; the operational scaling characteristics at large N are not demonstrated and are explicitly out of scope.

The cross-container partition-tolerance scenario is captured in a parallel artefact, not in the matrix of §5.4. The natural cross-container equivalent of the in-process scenario in §5.5 — stopping a container, allowing the running cluster to continue accepting work in its absence, restarting the container, and watching it rehydrate from its mounted volume and rejoin via catch-up against a surviving peer — is implemented in the same orchestrator that produced the matrix (the `--rehydrate-demo` flag of the demo script) and depends on no substrate feature that is not already exercised by the in-process test in §5.5. The decision to keep §5.4 to the four cells of the matrix, rather than expanding to a fifth cell, was rhetorical: the in-process exercise establishes the property structurally. The cross-container capture documents the same property under a different process boundary; it does not add a new claim.

The compaction ratio between inline and parametric workloads depends on Director rotation. Each new Director journals a fresh Define + Invocation pair, so the Define cache compacts on the Stage axis rather than on the journal axis. The compaction is real and measured at three rounds; its precise value at larger workloads is treated in Paper 2.

What would falsify the structural permission. The structural permission framing of §6 absorbs many objections by deferring to engineering work downstream of architecture — operational difficulties at scale, transport-specific failure modes, deployment-environment quirks. None of these would refute the structural permission; they would calibrate the cost of exercising it. The structural permission is refuted only by failure modes that show the substrate cannot host the deployment shape even in principle. This subsection names four such conditions.

- **A runtime central component is required for intra-cluster operation at $N > 3$.** If extending the §5 demonstration to N=4 or higher required introducing a process that must remain alive — a coordinator, a directory server, a bootstrap re-issuer — beyond the peers themselves, the cluster's peer-symmetric character would not generalize from N=3, and the appliance and peer-mesh shapes would not be structurally compatible

with cooperation beyond the demonstration's scale.

- **Journal-determinism breaks under transport change.** The substrate's claim of transport pluggability requires that the journal records (and replays) the same operations regardless of which transport carried them. If swapping Kestrel/HTTPS for an alternative transport — SimpleX, analog transfer, or any other — produced journals with different content for the same operations, or state not recoverable from replay, the transport-pluggability claim of §6 would be refuted.
- **Raw I/O in verb bodies systematically breaks journal replay.** §2 describes cloud-as-library as the substrate's idiomatic path, with raw I/O inside the verb body permitted as domain code under a known lock-cost penalty as the substrate's structural disincentive. If such I/O could systematically produce journal entries that the substrate could not replay deterministically — beyond the lock-cost penalty and the convention boundary of using Reactions or Tells for external invocation — the substrate's contribution to cloud-as-library would reduce from substrate-supported to purely convention-mediated, and Paper 2's "journal is the program" claim would be similarly narrowed.
- **Peer-mesh requires global identity for cluster-to-cluster operation.** If two journaled clusters established independently — each through analog bootstrap (§5.3) — could not be reconciled without consulting a shared identity authority, the peer-mesh's independence from central authority would not extend to inter-cluster operation, and the deployment-shape claims of §6 would narrow.

Each condition is testable. The paper does not undertake these tests; it states the structural permission and the conditions under which empirical evidence would refute it.

The lens of §7 extends to other architectural roles that admit the same analysis. Service discovery, queue-as-glue between bounded contexts, configuration-management roles, and distributed-tracing collectors are auditable under the two indicators; the present experiment does not exercise them, and the substrate's relation to each is left as a forward question. The lens admits this open extension by design — its diagnostic value is in reading which categories in any given system are contingent, not in enumerating them exhaustively.

9. Conclusion

This paper observed what happens to the conventional cloud-microservice stack under a journaled-program substrate developed across the prior six papers of this series: the datacenter ceases to be a structural requirement of running production software. Cloud and microservice ecosystems, when accessed from a journaled program, become libraries the program invokes rather than habitats the program must inhabit. Software becomes shippable as a physical artifact —

an appliance the customer owns, sized to the customer's domain, not a datacenter the vendor leases on the customer's behalf.

The observation was made operationally inspectable by porting the Ordering bounded context of `dotnet/eShop` to the substrate of §4, deploying it across three nodes that bootstrap each other through paper QR codes, and observing that after bootstrap the cluster operates against its own journals — none of the services that originally constituted the Ordering microservice are reachable from it, and the cluster continues under each one's absence. The demonstration does not argue uniqueness of the substrate; it establishes that the observation is inspectable in one working case.

The practical consequence (§6) is, as structural permission, an opening in the space of deployment shapes the conventional stack does not permit. Software becomes structurally compatible with delivery as an on-premises appliance, with cooperation across a peer mesh of devices (workstations, phones) without a central server, and with hybrid topologies that invoke cloud services as libraries. The shapes are not new categories — existing systems (Odoo Community, Sage 50, QuickBooks Desktop, Splunk Enterprise, Nextcloud) already deliver business software without a datacenter — but the journaled substrate offers a different operational profile that does not require the operator to compose specialized expertise (database administration, application hosting, backup, updates, orchestration) on top of the architecture. The paper establishes the structural permission; whether the permission becomes operational practice in any of these shapes is engineering work the paper does not undertake.

A side observation, stated honestly, is that the role prior literature names *server* becomes ephemeral under this substrate: discharged once per peer during analog bootstrap, then absent from the running topology. The construct *accidental category* (§7), applied at the role layer as an extension of Paper 6's *infrastructural symptom*, is offered as a diagnostic lens the reader may apply to their own architecture. The construct is the tool; the practical observation is the contribution.

The lens does not promise universal dissolution. Identity authorities, payment networks, sensor integrations, and computationally intensive services remain structural under any substrate. The reading is per role-in-use-case, not per system. What the lens offers is examinability: any production architecture can be examined role-by-role, asking whether each role is tied to the problem being solved or to the persistence model chosen.

Paper 6 applied the lens at the layer level, identifying many infrastructural layers — caches, queues, locks, application servers, orchestration clusters — as compensations for the underlying persistence model. The present paper observes the joint operational consequence: with those layers absent, and the architectural roles they serve revealed as accidental, the datacenter itself becomes optional, and the construction of production software passes back into the hands of those who model the domain. From this point on, treating the datacenter as a struc-

tural requirement is no longer an assumption but a choice that can be made explicit and evaluated.

Appendix A — Reproducibility

The artifact described in §5 is publicly available. The references in this appendix to file paths, test method names, and orchestrator scripts resolve against the released code; the artifact comprises the substrate framework, the runtime, the ported `Ordering` domain library and its test suite, the demo orchestrator, the supporting reports, and the asciinema recordings, hosted across two repositories described in §A.1.

A.1 Source and suite

The artifact is hosted in two public repositories under https://github.com/alvaroNCubo. The substrate framework (`Choreography/`), the runtime (`Puppeteer/`), the ported `Ordering` domain library and its test suite (`tests-local/UnitTestPaper7EShop/`), the demo orchestrator (`docker/`), and the supporting reports referenced in §A.7 (`notes/paper7_phase1_results.md`, `notes/paper7_phase2_results.md`) are in https://github.com/alvaroNCubo/puppeteer. The asciinema recordings and GIFs of §A.8 are in the `paper7-assets/` directory of https://github.com/alvaroNCubo/puppeteer-papers. Inline code citations of the form `file.cs:NN` resolve against the commit pinned in Code Provenance.

A.2 Demo orchestrator

The orchestrator script `docker/run-demo.sh` provisions three Docker containers, runs the analog bootstrap handshake against each, and exercises one cell of the matrix per invocation. Three flags select the scenario:

- `bash docker/run-demo.sh` — cross-container inline (22 entries final).
- `bash docker/run-demo.sh --parametric` — cross-container parametric (7 entries final).
- `bash docker/run-demo.sh --rehydrate-demo` — captures the cross-container leave-and-rejoin scenario referenced in §8: stops one container, allows the cluster to advance, restarts the container, and observes its rehydration and catch-up.

Each invocation runs in approximately 60–90 seconds on Docker Desktop.

A.3 Tests cited

The empirical claims in the body are exercised by specific tests in the suite:

Body section	Property	Test method	Project
§5.3	F1–F5 bootstrap handshake under real cryptography	`EndToEnd_RealCryptography_FullBootstrapHandshakeAndOnboardingTests`	`UnitTestPaper7EShop/TwoNodeOnboardingTests`
§5 (single peer operates alone)	Force-promoted Stage operates without peers	`SingleStage_OperatesAlone_WhenPromotedWithNoPeers`	`UnitTestPaper7EShop/ThreeNodeOrderingTest`
§5.4 (in-process, inline)	Three-stage Director rotation, inline regime	`ThreeStages_DirectorRotation_InlineRegime_HappyPath`	`UnitTestPaper7EShop/ThreeNodeOrderingTest`
§5.4 (in-process, parametric)	Three-stage Director rotation, parametric regime	`ThreeStages_DirectorRotation_ParametricRegime_HappyPath`	`UnitTestPaper7EShop/ThreeNodeOrderingTest`
§5 (rotation under cancellation)	Cancellation branch replicates across three stages	`CancellationBranch_ReplicatesAcrossThreeStages`	`UnitTestPaper7EShop/ThreeNodeOrderingTest`
§5.4 (instrumentation)	DLL load + per-phase convergence + journal bytes snapshot	`Metrics_F1_Snapshot`	`UnitTestPaper7EShop/ThreeNodeOrderingTest`
§5.5 (partition tolerance)	Leave-and-rejoin from disk, peer-to-peer catch-up	`OneStage_Offline_UntilAdvanced_RejoinsAndCatchesUpViaPeer`	`UnitTestPaper7EShop/ThreeNodeOrderingTest`
§5.5 (rehydration without onboarding)	Rehydrated Stage restores identity from disk alone	`RehydratedStage_RestoresIdentityFromDisk_WithoutOnboarding`	`UnitTestPaper7EShop/ThreeNodeOrderingTest`

A.4 Cryptographic stack

The cryptographic primitives underlying the F1–F5 handshake of §5.3 are documented in `Choreography/Usher/Crypto/*.cs`. The implementations use BouncyCastle 2.6.2:

- Ed25519 keypair generation, signing, verification: `Ed25519StageKeyGenerator.cs`, `Ed25519Signer.cs`, `Ed25519SignatureVerifier.cs`.
- Ed25519-to-X25519 derivation (Edwards-to-Montgomery, $u = (1 + y) / (1 - y) \bmod p$): `Ed25519ToX25519.cs`.
- Sealed-box AEAD using X25519 ECDH and ChaCha20-Poly1305, with the participating public keys bound as additional authenticated data: `SealedBoxPayloadSealer.cs`.

- Kestrel TLS listener and self-signed certificate generation with SAN: `HttpsTransportListener.cs`, `SelfSignedCert.cs`.
- TOFU certificate fingerprint pin via `SocketsHttpHandler.SslOptions.RemoteCertificateValidation` `HttpsClientFactory.cs`.

A.5 Share-link URI

The share-link emitted by the issuer takes the form `puppeteer-usher://v1/{base64url(json)}`. The JSON payload carries the transport address of the issuer endpoint and a SHA-256 fingerprint (`fp` field) of its TLS certificate. The encoder/decoder is defined by `IUsherShareLinkEncoder` in `Choreography/Usher/`. A representative share-link under load is 449 characters in length, within the alphanumeric capacity of a QR code at version 14 with error correction level M (approximately 535 characters).

A.6 Wire format

The HTTPS transport between containers carries a `ChannelPurpose` byte after the sender identifier in each frame. This wire format (v2) prevents Define and Invocation channels from collapsing into the same channel-dictionary key when Director rotation runs the parametric workload across multiple containers; the in-process regime is unaffected. The wire format is implemented in the transport handlers under `Choreography/Transport/Https/`.

A.7 Supporting reports

Two markdown reports in the `notes/` directory of the same branch document the experiment in detail:

- `notes/paper7_phase1_results.md` — the in-process row of the matrix (eleven sections plus three addenda).
- `notes/paper7_phase2_results.md` — the cross-container row, the hardening cycle that converged on the demo, the three-Docker mesh, the rotation protocol, and the parametric cell.

A.8 Recordings

Eleven cast recordings (asciinema format) and eleven GIF renders capture the four cells of the matrix of §5.4 and the rehydration property of §5.5, one recording per Docker container per cell. The recordings are published alongside this paper as supplementary material, hosted in the `paper7-assets/` directory of the `puppeteer-papers` repository.

Appendix B — Role-level diagnostic table

The lens of §7 — Paper 6's *infrastructural symptom* applied at the role layer — produces a reading per role-in-use-case. The same node, identified in a deployment as "API server," reads as an accidental category when deployed to accept

authoritative writes against a shared database, and as a structural requirement when deployed to enforce cryptographic identity claims. The discriminator is the function, not the name. Seven canonical categories of architectural role admit reading under the two indicators of the lens; their substrate-property counterparts are summarized below as the role-level analog of Paper 6's Table 1.

Table B.1. Diagnostic reading: seven canonical categories of architectural role under the two indicators of Paper 6's lens, re-expressed at the role layer.

Architectural role	Representational choice that necessitates it	What a substrate satisfying both indicators offers instead
Server as write authority	Nodes replicate data; consistency requires a single point of acceptance	Each node accepts writes locally; the journal records the program, which replays deterministically on every node
Master / primary in replication	Nodes replicate state diffs; a primary must order them	Each node owns its journal; cross-node causality is recorded by reference, not orchestrated
Coordinator in distributed transactions	Multiple writers contend for a logical entity across processes	Each logical entity has a single point of authority intrinsic to the substrate; serialization is not externalized
Stateful gateway	Clients cannot directly invoke deferred work without losing causal record	Deferred work is journal-recorded; the substrate carries causality, not the gateway
Application server / runtime container	The runtime, the framework, and the application require disparate substrates	A minimal substrate suffices to run the domain artifact
Bootstrap service	New nodes require an issuer of credentials and addresses	Bootstrap is information transfer that crosses by any means, including analog artifacts; the issuer process is ephemeral

Architectural role	Representational choice that necessitates it	What a substrate satisfying both indicators offers instead
Orchestrator as coordination substrate	Stateful coordination across instances requires external choreography	Coordination is between actors; the substrate provides location and continuity intrinsically

The seven rows are not exhaustive; service discovery, queue-as-glue between bounded contexts, and configuration management admit similar analysis. The lens is diagnostic, not prescriptive: it enables auditing existing architectures under the two indicators of Paper 6, independently of any decision to adopt the substrate of §4.

Code provenance

Source-code references in this paper resolve against the public Puppeteer repository at commit `b42d0f7` (`2026-05-26`). This commit includes the substrate code described in §4-§5, the reproducibility artifacts of Appendix A (`docker/`, `notes/`), and the SimpleX-based transport implementation referenced in §6 (`Choreography/Transport/SimpleX/`). Earlier commits in the repository preserve the substrate's prior states: `91118fc` (2026-05-22) added the reproducibility artifacts; `2f31f96` (2026-05-18) is the earlier substrate snapshot before the SimpleX transport fixes (C2S envelope decryption, batch parsing, catch-up piggyback routing, StageManager Director-rotation race, Usher F5 invitation-consumed race) of commit `b42d0f7`. The earlier substrate snapshot is archived in Software Heritage under the following persistent identifier:

```
swh:1:dir:10e7e6bad7eb77b6c2e406762026177f95c3ae92;
  origin=https://github.com/alvaroNCubo/puppeteer;
  anchor=swh:1:rev:2f31f9674a5de816bdf1bf9d8360ff218a02e4da
```

Inline references of the form `file.cs:NN` (e.g., `ActorHandler.cs:38`) resolve against `b42d0f7`. A reader can construct a per-file SWHID by adding the qualifiers `;path=<path>;lines=<NN>` to the directory SWHID above for the `2f31f96` snapshot; for the post-`2f31f96` state (which includes the SimpleX transport fixes of `b42d0f7`), the GitHub `b42d0f7` reference is canonical. Future commits to the repository may renumber lines; the SWHID preserves the cited `2f31f96` state independently of any future change to the repository or its hosting.

Acknowledgments

The author used large language models (including Claude and ChatGPT) as editorial assistants for language refinement, structural feedback, and literature navigation. All original ideas, terminology, theoretical constructs, and technical content presented in this work are solely the author's.

References

Ameloot, T. J., Neven, F., & Van den Bussche, J. (2013). Relational transducers for declarative networking. *Journal of the ACM*, 60(2), 1–38.

Armstrong, J. (2003). *Making reliable distributed systems in the presence of software errors* [Doctoral thesis]. KTH Royal Institute of Technology.

Bernstein, P. A., Bykov, S., Geller, A., Kliot, G., & Thelin, J. (2014). *Orleans: Distributed virtual actors for programmability and scalability* (Tech. Rep. MSR-TR-2014-41). Microsoft Research.

Brewer, E. A. (2000). Towards robust distributed systems [Keynote address]. *Proceedings of the Nineteenth Annual ACM Symposium on Principles of Distributed Computing (PODC '00)*.

Cognitect. (n.d.). *Datomic: A transactional database with a flexible data model, queryable as a log of changes* [Software project]. Retrieved May 22, 2026, from https://www.datomic.com/

DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Vosshall, P., & Vogels, W. (2007). Dynamo: Amazon's highly available key-value store. *Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP '07)*, 205–220.

Earthstar Project. (n.d.). *Earthstar: simple personal data sync* [Software project]. Retrieved May 22, 2026, from https://earthstar-project.org/

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Event Store Ltd. (n.d.). *EventStoreDB: The database for event sourcing* [Software project]. Retrieved May 23, 2026, from https://www.eventstore.com/eventstoredb

Fowler, M. (2005). Event sourcing. Retrieved from https://martinfowler.com/eaaDev/EventSourcing.html

Gilbert, S., & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2), 51–59.

Gregor, S. (2006). The nature of theory in information systems. *MIS Quarterly*, 30(3), 611–642.

Harris-Braun, E., Luck, N., & Brock, A. (2018). *Holochain: Scalable agent-centric distributed computing* (Alpha 1 white paper). Holo. https://www.holochain.org/documents/holochain-white-paper-alpha.pdf

Hellerstein, J. M. (2010). The declarative imperative: Experiences and conjectures in distributed logic. *ACM SIGMOD Record*, 39(1), 5–19.

Hewitt, C., Bishop, P., & Steiger, R. (1973). A universal modular ACTOR formalism for artificial intelligence. *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI)*, 235–245.

Holepunch. (n.d.). *Hypercore: append-only logs for the peer-to-peer web* [Software project]. Retrieved May 22, 2026, from https://docs.pears.com/building-blocks/hypercore

Kleppmann, M., Frazee, P., Gold, J., Graber, J., Holmgren, D., Ivy, D., Johnson, J., Newbold, B., & Volpert, J. (2024). Bluesky and the AT Protocol: Usable decentralized social media. In *Proceedings of the ACM CoNEXT-2024 Workshop on the Decentralization of the Internet*. ACM. https://doi.org/10.1145/3694809.3700740

Kleppmann, M., Wiggins, A., van Hardenberg, P., & McGranaghan, M. (2019). Local-first software: You own your data, in spite of the cloud. In *Onward! 2019: Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software* (pp. 154–178). ACM. https://doi.org/10.1145/3359591.3359737

Lemmer-Webber, C., Tallon, J., Shepherd, E., Guy, A., & Prodromou, E. (2018). *ActivityPub* (W3C Recommendation, 23 January 2018). World Wide Web Consortium. https://www.w3.org/TR/activitypub/

Lightbend. (n.d.). *Akka Persistence* [Software documentation]. Retrieved May 23, 2026, from https://doc.akka.io/libraries/akka-core/current/typed/persistence.html

Matrix.org Foundation. (n.d.). *Matrix specification* [Online document]. Retrieved May 22, 2026, from https://spec.matrix.org/

Microsoft. (2026). *dotnet/eShop: A reference .NET application implementing an eCommerce site* [Software repository, commit 9b4f943, 2026-04-21]. Retrieved May 22, 2026, from https://github.com/dotnet/eShop

Nygard, M. T. (2018). *Release it! Design and deploy production-ready software* (2nd ed.). Pragmatic Bookshelf.

Ogden, M., McKelvey, K., & Madsen, M. B. (2017). *Dat: Distributed dataset synchronization and versioning* [White paper]. Code for Science. https://github.com/datprotocol/whitepaper

Ongaro, D., & Ousterhout, J. (2014). In search of an understandable consensus algorithm. *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC '14)*, 305–319.

Rivera, A. (2026a). Anti-porous architecture: a unified design principle for CQRS + Actor + Event-Sourcing systems. *Puppeteer Papers Series*, Paper 1. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/01-anti-porosity.md

Rivera, A. (2026b). Program–value separability: the structural precondition for compilation, caching, and dense journaling in a DSL runtime. *Puppeteer Papers Series*, Paper 2. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/02-program-value-separability.md

Rivera, A. (2026c). Reactions and the partition: opt-in eventual consistency in actor-native systems. *Puppeteer Papers Series*, Paper 3. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/03-reactions-and-partition.md

Rivera, A. (2026d). Preserving semantic continuity across actors: a tell-based approach without orchestration. *Puppeteer Papers Series*, Paper 4. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/04-cross-actor-continuity.md

Rivera, A. (2026e). The journal as substrate: unifying deployment, replication, backup, and offline operation in distributed systems. *Puppeteer Papers Series*, Paper 5. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/05-substrate-operations.md

Rivera, A. (2026f). Most infrastructure layers are symptoms of the persistence model: a construct for auditing production stacks. *Puppeteer Papers Series*, Paper 6. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/06-infrastructural-symptom.md

Sambra, A. V., Mansour, E., Hawke, S., Zereba, M., Greco, N., Ghanem, A., Zagidulin, D., Aboulnaga, A., & Berners-Lee, T. (2016). *Solid: A platform for decentralized social applications based on linked data* (Technical Report). MIT CSAIL & Qatar Computing Research Institute. http://emansour.com/research/lusail/solid_protocols.pdf

Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). *Conflict-free replicated data types* (Research Report RR-7687). INRIA.

Tarr, D., Lavoie, E., Meyer, A., & Tschudin, C. (2019). Secure Scuttlebutt: An identity-centric protocol for subjective and decentralized applications. In *Proceedings of the 6th ACM Conference on Information-Centric Networking (ICN '19)* (pp. 1–11). ACM. https://doi.org/10.1145/3357150.3357396

Vernon, V. (2013). *Implementing domain-driven design*. Addison-Wesley.

Vernon, V. (2015). *Reactive messaging patterns with the actor model: Applications and integration in Scala and Akka*. Addison-Wesley.

Young, G. (2010). *CQRS documents*. Retrieved from https://cqrs.files.wordpress.com/2010/11/cqrs_document